# Functions in Python

The whole function toolkit on one page: define, call, pass, return, scope, document.



## Template

```python
def greet(name: str, *, shout: bool = False) -> str:
    """Return a greeting for *name*."""
    msg = f"Hello, {name}"
    return msg.upper() if shout else msg

print(greet("Ann"), "|", greet("Ann", shout=True))
```

```
OUTPUT
Hello, Ann | HELLO, ANN
```

## I want to...

| Goal | Write |
|---|---|
| define a function | `def f(x):` |
| give a parameter a default | `def f(x, n=1):` |
| call with names | `f(x=1, n=2)` |
| accept any # of positionals | `def f(*args):` |
| accept any # of keywords | `def f(**kwargs):` |
| unpack into a call | `f(*seq, **mapping)` |
| force positional / keyword | `def f(a, /, b, *, c):` |
| return several values | `return a, b` |
| make an anonymous function | `lambda x: x + 1` |
| rebind an outer variable | `global x` / `nonlocal x` |
| document it | docstring + `def f(x: int) -> str:` |
| inspect it | `help(f)`, `inspect.signature(f)` |

## Scope: LEGB

Names resolve **L**ocal → **E**nclosing → **G**lobal → **B**uilt-in. Assigning inside a function makes the name local unless declared `global` / `nonlocal`.

## Function objects have attributes

```python
def f(a, b=2):
    """Demo."""

print(f.__name__, f.__defaults__, f.__doc__)
print(f.__code__.co_varnames)
```

```
OUTPUT
f (2,) Demo.
('a', 'b')
```

## print is not return

```python
def add(a, b):
    print(a + b)          # shows 3 but returns None

x = add(1, 2)
print(x)
```

```
OUTPUT
3
None
```

## Five classic bugs

| Bug | Fix |
|---|---|
| forgot `return` → `None` | return the value (don't just `print` it) |
| mutable default `def f(x=[])` | `x=None`, then create inside |
| assign to a global → `UnboundLocalError` | pass in + return, or declare `global` |
| `f` vs `f()` | `f` is the object; `f()` calls it |
| lambda in a loop sees final value | bind with `lambda i=i: ...` |

## Design rules

- One job per function; name it with a verb (`load_users`, `is_valid`).
- Aim for ≤ 3–4 parameters; group related ones in a dict or dataclass.
- Prefer pure functions; raise exceptions instead of returning error codes.
- Return results – let the caller decide to print, save or send.

**REMEMBER** A function is an object: it can be assigned, passed around, returned, stored in collections and decorated.